A Needle in a Data Haystack – 678978 – Final Project Report

Password Strength Analysis Dashboard

Team members

[Here were names, usernames and ids]

Streamlit: Password Strength Analysis Dashboard
Demonstration Video: YouTube Video
GitHub: GitHub Code

Problem Description

The increasing frequency of data breaches highlights the vulnerability of commonly used passwords. Weak and predictable passwords make users susceptible to attacks, with hackers easily exploiting them through brute-force techniques or dictionaries of popular passwords. The goal of our project is to create an interactive dashboard that provides users with a comprehensive analysis of their password strength. This analysis uses entropy, clustering algorithms, and statistical models to assess the predictability and security of passwords. The dashboard not only evaluates the password's current strength but also offers actionable suggestions for improvement.

Our project focuses on answering the following key questions:

1. What common patterns do people use in password creation?
2. How can we quantify the strength of a password?
3. What guidance can we offer users to help them create stronger, more secure passwords?

Datasets

For this project, we utilized several large datasets comprising both breached passwords and common word lists. These datasets were essential for evaluating password strength, identifying common patterns, and developing cracking algorithms:

1. Rockyou2024: The most recent and largest collection of passwords, containing 10 billion entries along with word dictionaries. Requires cleaning and ordering. 45GB.
2. Pwnd Passwords: A dataset of 500 million passwords hashed with MD5 and NTLM, including their frequency counts. Serves as our test set for evaluating our cracking capabilities. 10.3 GB
3. Antipublic breach dataset: About 120 million passwords from various breaches. 2 GB.
4. RockYou: 14 million passwords with counts from the 2009 RockYou breach. 174 MB.
5. Pwdb 100K Top Passwords: The 100,000 most common passwords with their counts.

Our Solution - Password Analysis dashboard

We implemented a wide range of methodologies and provided users with dynamic visualizations that make password strength assessment both engaging and informative. The dashboard was built using Streamlit, a Python-based framework for creating interactive web applications.

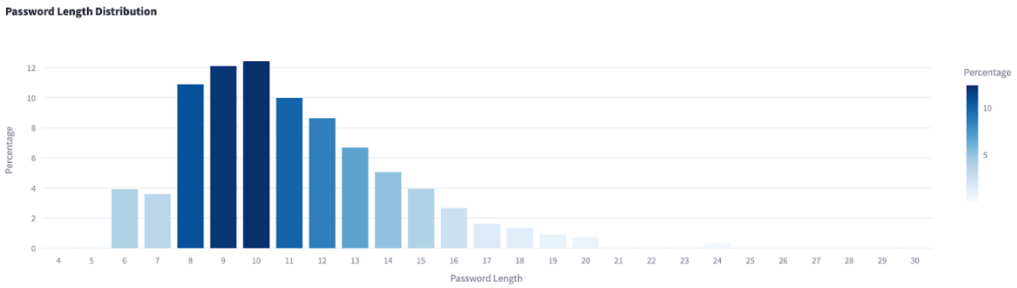
General Password Analysis – Breached Password Analysis Page in Dashboard

Through the whole project, the analysis focuses on the Rockyou 2024 dataset, which contains over 10 billion passwords. To make the analysis more manageable with the available tools, we randomly selected 1 million passwords as a representative sample. Minimal preprocessing was applied, filtering out passwords longer than 30 characters to better reflect human-created passwords, and retaining only those containing ASCII characters. After reviewing various password breach datasets, we determined that this dataset was the most relevant for modern websites' password requirements, as it featured longer passwords with a wider range of character types (uppercase letters, numbers, and special characters), which older datasets lacked. Even with this more secure dataset, we uncovered significant insights into common password creation patterns, revealing that many passwords remain insecure. The visualizations of this dataset provided valuable insights into these behaviors, helping

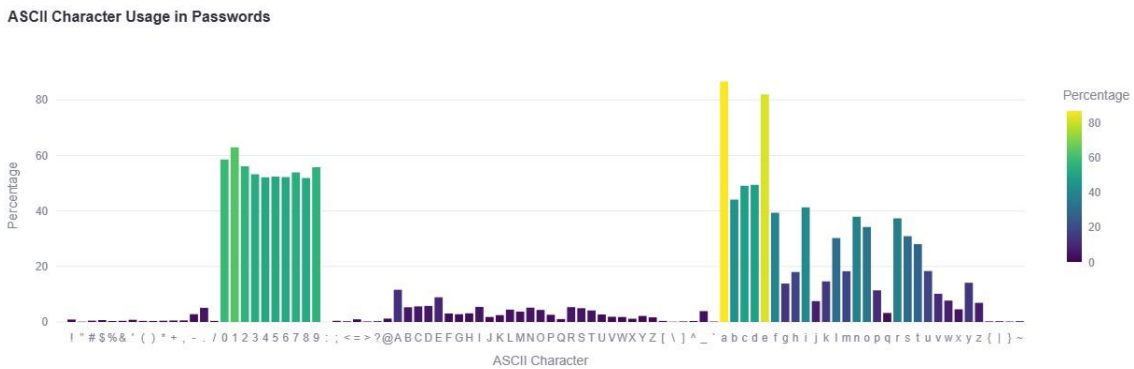
users recognize patterns that may compromise password strength. The dashboard includes a variety of visualizations to help users understand password patterns and security risks. A **Password Length Distribution** bar chart illustrates the percentage of passwords by length, revealing that certain lengths are more common, with extremes being less prevalent. The **ASCII Character Usage** bar chart highlights the frequent use of lowercase letters and the underutilization of special characters, suggesting areas for improvement. A **Password Categories Distribution** treemap shows how users tend to favor simpler combinations, like "Lowercase Only" or "Uppercase and Numbers," with larger blocks representing more common categories. The **Number Position Violin Plot** visualizes where numbers are placed, often at the beginning or end of passwords, which can indicate weaker security patterns. Similarly, the **Special Character Position Violin Plot** reveals that special characters are typically placed in the middle of passwords. The **Year Usage** bar chart shows the frequency of specific years in passwords, often birth years, highlighting how personal information can compromise security. An **Entropy Distribution** histogram helps users assess password strength by measuring unpredictability, ranging from weak (low entropy) to strong (high entropy) passwords. **Average Character Usage by Password Length** bar charts illustrate how the number of numbers, uppercase, lowercase, and special characters varies across different password lengths. Finally, **Character Counts for Specific Password Length** interactive bar chart allows users to input specific lengths and see the average counts for various character types.

Insights Revealed in our analysis

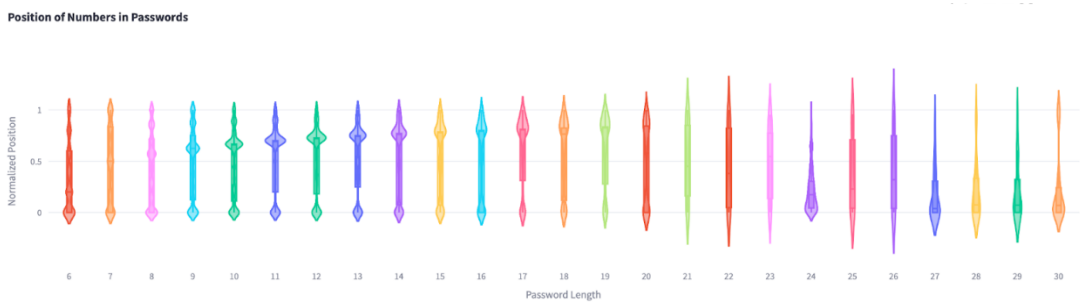
- **Password Length Distribution:** Our analysis revealed a curve-like distribution of password lengths, indicating a "sweet spot" that most users gravitate toward. Shorter and longer passwords were less common, forming the tails of the distribution. This pattern provides insights into user behavior and vulnerabilities in password selection.

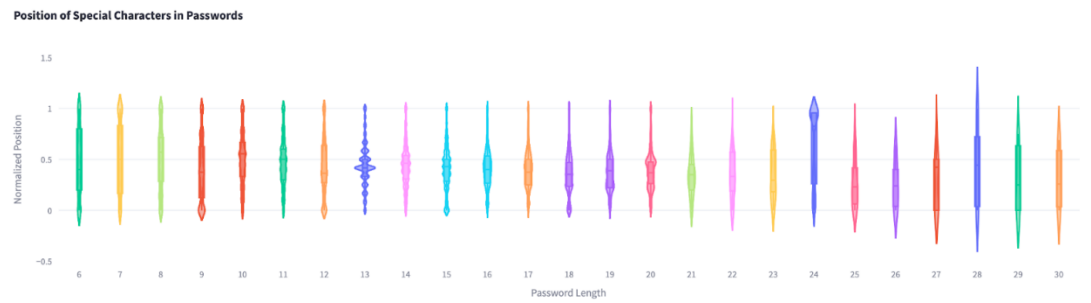


- **Character Composition:** Lowercase letters were overwhelmingly popular, while special characters like '?' or '>' were used less frequently, indicating an opportunity to improve password security by promoting their use. The most frequent password category combined lowercase letters and numbers, followed by passwords containing only lowercase letters. More complex combinations, such as those incorporating uppercase letters, special characters, and numbers, were relatively rare, highlighting the need for stronger password creation habits.

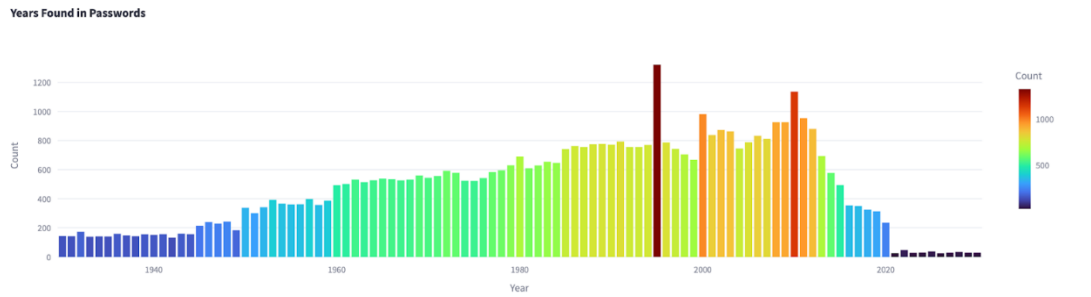


- **Number and Special Character Placement:** Numbers were often placed at the beginning or end of passwords, while special characters (e.g., from the set: !@#$%^&*()-_=+[]{|;:',.<>?/`~) were more likely to appear in the middle. These patterns remained consistent across different password lengths, revealing common habits in password creation.

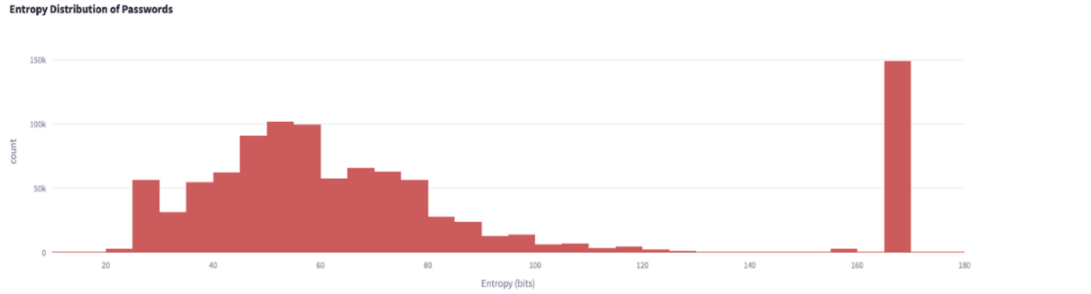




- **Year Usage:** Many passwords included years, likely representing birth years or significant dates. Years between 1960-2010 were most common, likely reflecting the age distribution of internet users. This trend highlights how personal information often influences password creation.



- **Entropy and Password Strength:** By calculating entropy, we measured the unpredictability and strength of passwords. The distribution formed a curve, with most passwords falling in the middle range of complexity. Very weak and very strong passwords were less common, but a notable spike in high-entropy passwords suggested the presence of randomly generated passwords, possibly from password managers.

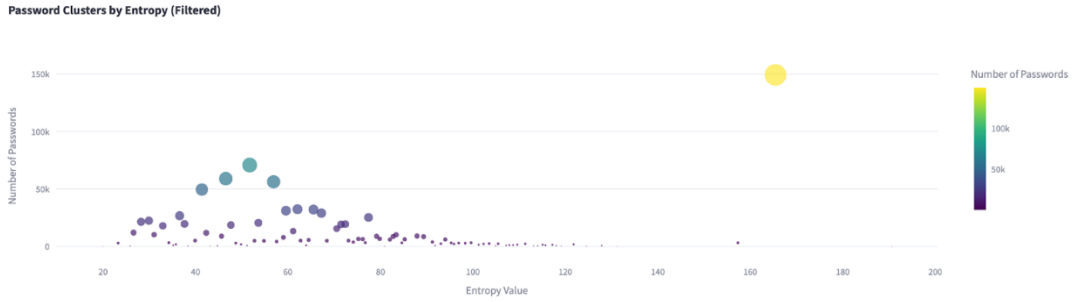


Clustering Password Analysis - Breached Password Analysis Page in Dashboard

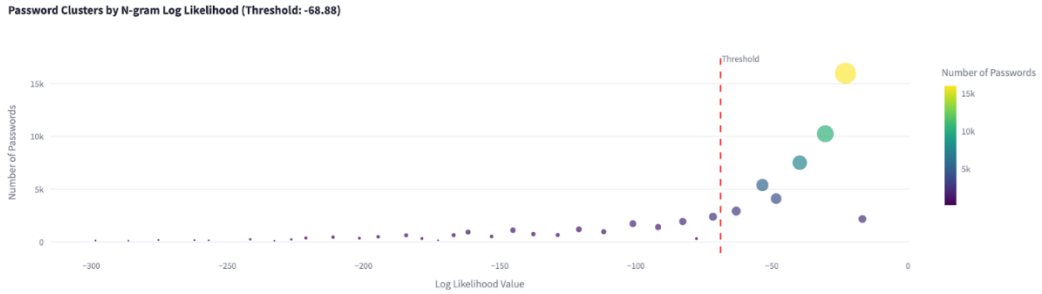
Our clustering analysis aimed to categorize passwords based on various metrics, providing insights into their predictability, security, and structural patterns.

Clustering by entropy

Entropy is a fundamental measure in password security, representing the randomness or unpredictability of a password. It is calculated based on both the length of the password and the diversity of characters used, such as lowercase and uppercase letters, numbers, and special symbols. A larger character set increases the number of potential combinations, making the password more secure. We calculated entropy for each password using the formula: Entropy = Password Length × $\log_2$(Character Set Size). Passwords with an entropy difference of less than 0.5 were grouped into clusters. For users, we provided a visual representation of which entropy cluster their password belonged to, along with examples of similar passwords. This helped users understand the complexity of their password and encouraged them to improve entropy by increasing password length and incorporating diverse characters.



Clustering by N-gram log-likelihood



N-gram log-likelihood is a statistical technique that evaluates the probability of a sequence of characters appearing within a password based on trained language models. This method is particularly useful for detecting passwords that follow predictable linguistic or character sequence patterns, which may render them more vulnerable to attacks. While entropy measures the randomness and complexity of a password by considering factors such as length and character diversity, it does not account for semantic patterns or common linguistic structures that may render a password more susceptible to guessing. For example, the password **"WelcomeBack2022!"** may exhibit high entropy due to its inclusion of uppercase and lowercase letters, numbers, and special characters. However, it consists of common words and a predictable sequence, which results in a higher n-gram log-likelihood. The higher log-likelihood indicates that the password aligns closely with popular password patterns and is therefore more meaningful and potentially more vulnerable to attacks that exploit linguistic predictability. The n-gram model captures the likelihood of observing an n-character sequence (e.g., bigrams or trigrams) by analyzing large corpora of text or, in our case, password data. For this analysis, we utilized **bigrams** (2-character sequences) to calculate log-likelihood scores, allowing us to cluster passwords that exhibit similar linguistic patterns.

Implementation steps:

1. **Meaningful Dataset:** We built the language model by calculating bigram probabilities using the **RockYou 2009** dataset, a well-known breach dataset containing millions of passwords. This dataset is particularly useful because it includes numbers and special characters, which are often present in passwords but not in typical natural language datasets. The bigram model captures frequently used sequences, such as “pa” (from "password") or common numeric sequences like "12", which are more prevalent in passwords than in regular text.
2. **Gibberish Dataset:** Next, we generated a gibberish dataset composed of randomly generated character sequences. Calculating the bigram probabilities for this dataset provides a contrasting baseline that represents uniformity over ascii characters and the absence of meaningful patterns.
3. **Calculating Bigram Log-Likelihood**: For each password in our analysis, we calculated its log-likelihood score by summing the logarithms of the bigram probabilities from RockYou 2009 model.
4. **Defining a Threshold:** We established a threshold to differentiate between meaningful and gibberish passwords. The threshold is calculated as the average between the highest log-likelihood value in the gibberish dataset and the lowest log-likelihood value in the meaningful dataset. This separation allowed us to classify passwords: those below the threshold were considered random or gibberish, while those above the threshold were classified as meaningful.
5. **Clustering Passwords Based on Log-Likelihood Scores:** Passwords with similar scores were placed into the same cluster, effectively organizing passwords based on how predictable they are from a linguistic perspective.

Clustering by MinHash

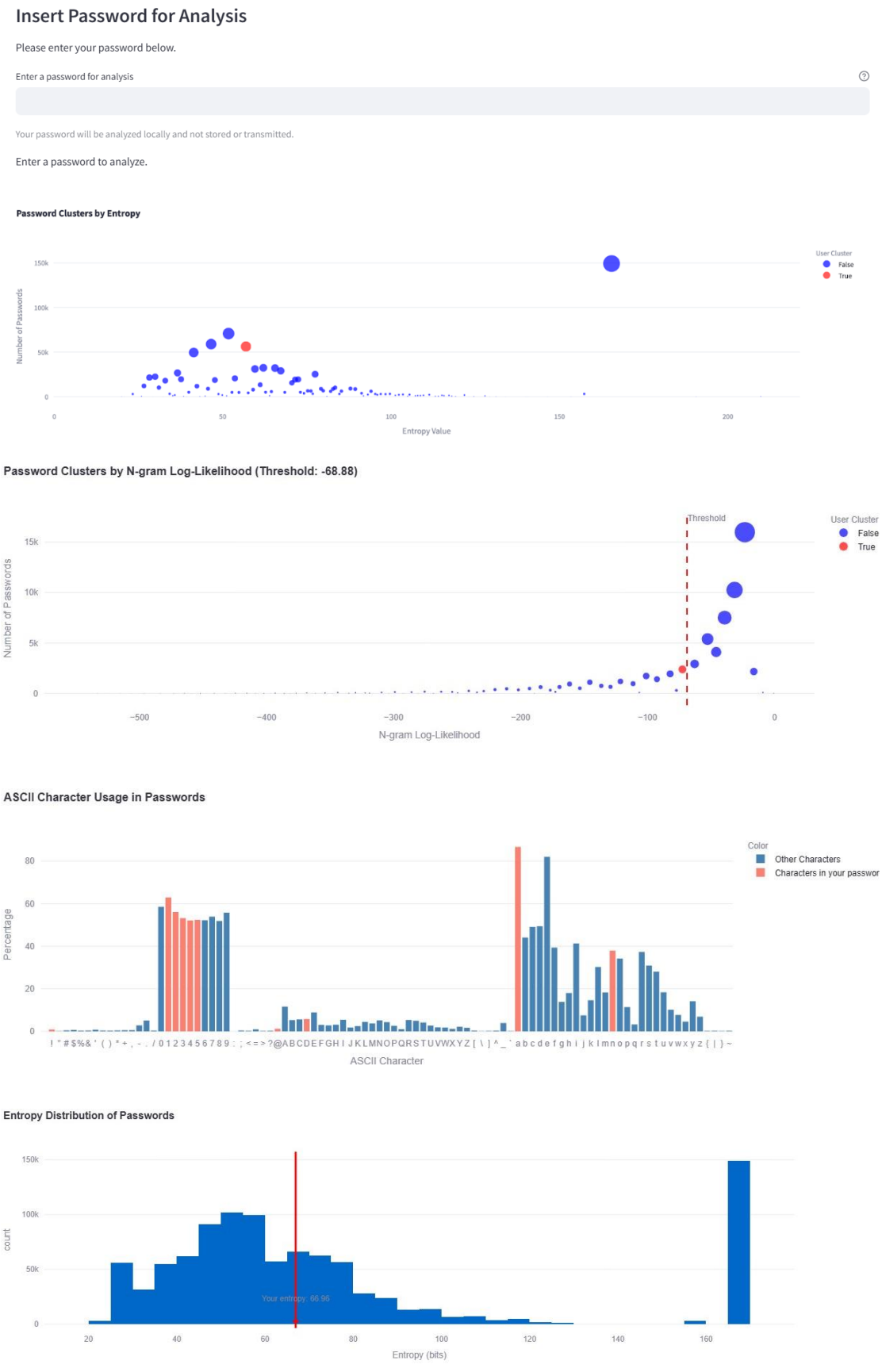
MinHash is a technique that efficiently estimates the similarity between sets of elements, making it well-suited for password clustering. By converting each password into a MinHash signature, we were able to compute approximate Jaccard similarities between passwords or clusters. This method allowed us to group structurally similar passwords, even when the order or specific characters differed. To help users better understand these clusters, we used **Multidimensional Scaling (MDS)** to project the high-dimensional similarity matrix onto a 2D plane. The result was a visual scatter plot where cluster sizes and distances between them indicated the similarity of passwords. Users could explore clusters, hovering over points to see example passwords and their commonalities.

<u>User Input Password Statistics and Strength Analysis</u>

The dashboard includes two pages which allow users to input their own passwords and receive real-time interactive analysis and feedback on strength and improvements.

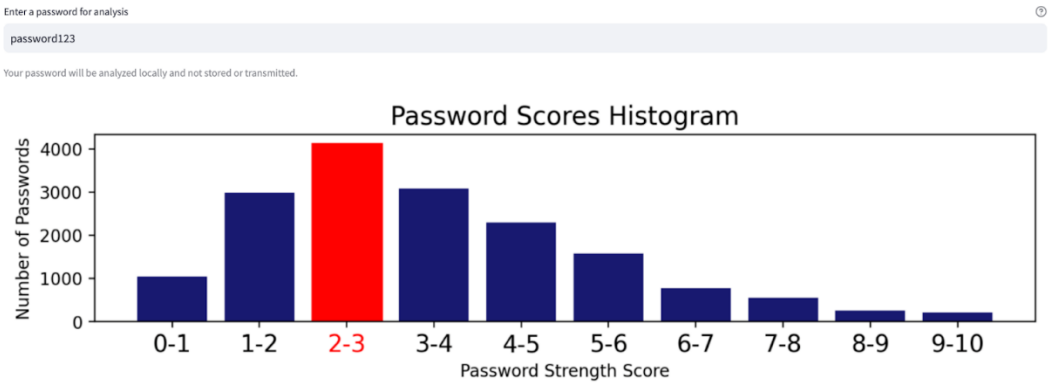
Statistics and Cluster Visualization - Check your Passwords Statistics Page

Users can input a password to see which cluster it fell into. The user's password was highlighted in red, while similar clusters are shown in blue. This helps users understand how common or unique their password is relative to others. The dashboard provides immediate feedback as users type in.



Our Password Strength Algorithm - Check your Passwords Strength Page

We define a heuristic algorithm to evaluate the strength of a given password. Our method examines basic features such as the password's length, number of unique characters, and use of different character types, as well as advanced techniques, such as regular expressions to detect common patterns and predefined lists to identify passwords containing popular words and numbers.



<u>Password Strength Algorithm Implementation steps</u>

Step 1 - Initial Disqualification Tests

When evaluating a password, we first run basic tests to quickly disqualify weak passwords, assigning them a score of 0. If the password meets any of these conditions, it is immediately given a score of 0, as such patterns are highly predictable and easily compromised. This initial screening involves checking five key conditions:

1. Common Passwords: The password appears in a predefined list of common passwords, numbers, popular words or names. These lists were compiled through an analysis of various datasets, common male, female, and family names. This approach allows us to detect passwords that are superficially complex but are actually based on easily guessable patterns.
2. Numeric Passwords: The password is shorter than 9 characters and consists only of numbers.
3. Repeated Characters: The password consists of a single character repeated multiple times.
4. Date Formats: The password matches a common date format (e.g., 24/07, 07-24, 2407, 24.07.1999, etc.).
5. Email Format: The password matches the user's email address format (e.g., xxxx@yyy.zz), which is considered highly insecure.

Step 2 - Detecting Fake Complexity

We proceed to analyze the password's true complexity. We create two additional variations of the password to detect "fake complexity," where a password may seem strong at first glance but is actually composed of common patterns that are vulnerable to brute-force attacks.

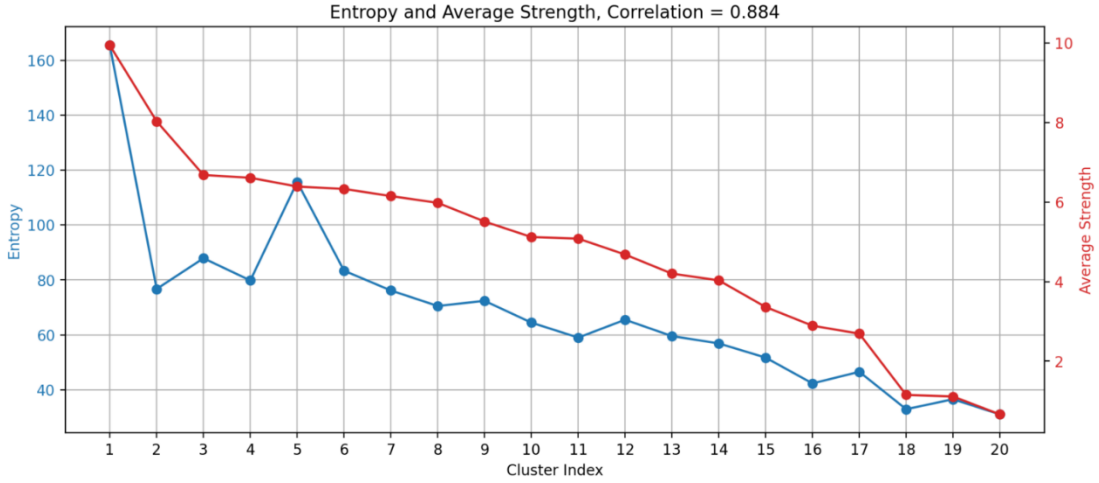
1. In the **first variation**, we remove punctuation symbols and sequences of digits, transforming the password into a recognizable word or phrase (if it exists in our predefined word lists).
2. In the **second variation**, we replace any detected word or number substring with a single character, based on the rationale that any distinct word serves as a single additional unit of complication, much like the complexity that is added by the addition of a single random character.

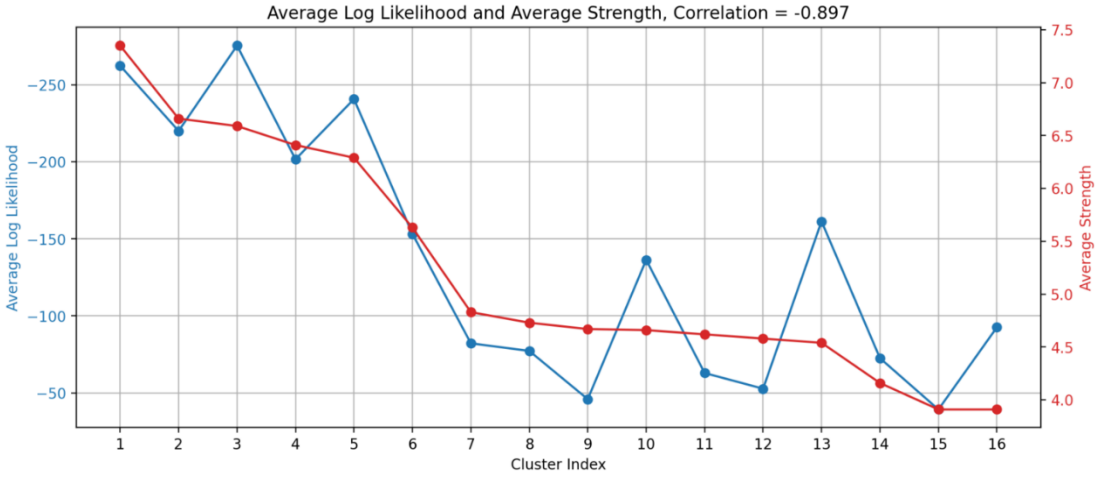
Step 3 - Final Strength Calculation

After analyzing the password and its two variations, we compute a strength score for each version based on its features, such as length, character diversity, and detected patterns. We then combine these scores using a weighted sum and normalize the result to a scale of 0 to 10, where **0** represents a password that can be cracked within seconds using basic brute-force techniques and **10** represents a highly secure password that does not exhibit any significant weaknesses.

To evaluate our algorithm, we selected 20 random clusters based on entropy and calculated the average entropy and password strength for each cluster. We found a strong positive correlation

between these two metrics, which aligns with expectations since higher entropy typically reflects greater unpredictability, a key trait of strong passwords. Additionally, we analyzed 16 random n-gram log-likelihood clusters and observed a strong negative correlation, indicating that passwords with higher log-likelihoods tend to be weaker and more predictable.





<u>Evaluation - Password Cracking</u>

To further assess the effectiveness of our analysis and recommendations, we developed a smart brute-force algorithm and tested it on the "Pwned" dataset. The "Pwned" dataset, which is widely trusted in the security community, contains over 500 million SHA-1 hashed passwords along with their frequency counts, collected from various breaches over the years. It is one of the largest and most reliable sources of real-world password data, making it an ideal benchmark for evaluating password security. While cracking a single strong password can take years, targeting a significant portion of a large database is feasible and is a common tactic in cyber threats. Our algorithm, leveraging brute-forcing techniques, carefully selected dictionaries, and statistical insights, was designed to crack as many passwords as possible within a reasonable timeframe.

Phase 1: Cracking Based on Simple Combinations

In the initial phase, we generated password candidates by combining common names (popular boys, girls, dog names, etc.) with common numeric sequences. These combinations were hashed and compared against the 10,000 most popular hashed passwords. By appending common passwords like "123456" and "12345" to names, we successfully cracked 33 distinct passwords. We found that appending numbers to the right of names yielded more results than appending them to the left, and shorter numeric sequences (e.g., "1") were more effective than longer ones (e.g., "123456"). Building on these findings, we expanded our approach to a larger dataset containing 2,826 popular names and explored several techniques. First, we cracked 1,145 passwords using direct name matching, achieving

an 11.45% success rate. Then, by appending single digits (1-9) to the names, we cracked an additional 797 passwords, resulting in a 7.97% success rate. Lastly, combining pairs of digits (0-9) without repetition yielded 275 cracked passwords, with a success rate of 2.75%.

Phase 2: Incorporating Patterns
In this phase, we incorporated common keywords—such as names and popular terms from social media, sports, and pop culture—into predictable structures that users often follow. These combinations were crafted to align with common password templates, such as sequences of digits or phrases like "ILoveMyMom." While these phrases may seem complex, they provide little protection against dictionary-based attacks. More sophisticated patterns included concatenating common words with digits or punctuation marks (e.g., "Hello123!") and words separated by special characters (e.g., "Name_123"). Using this approach, we successfully cracked 2,810 passwords out of a set of 10,000.

Phase 3: Large Scale Cracking
To further explore password vulnerabilities, we applied our approach to crack passwords using widely available leaked password dictionaries. By targeting the first 10 million passwords with the highest frequency counts, we successfully cracked over 8 million passwords using datasets such as the Antipublic breach, RockYou, and Pwdb 100K Top Passwords. These datasets contain hundreds of millions of commonly used passwords, many of which include frequency data. Although our cracking approach was straightforward, it effectively demonstrated how easily common patterns can be exploited. The ease with which we cracked such a large number of passwords underscores the dangers of relying on weak or commonly used passwords—especially when smart patterning techniques, like those we demonstrated in the earlier phases, are employed.

Future Work

Moving forward, our system can be enhanced by incorporating real-time data from newly leaked password databases across different regions and platforms. Expanding beyond the datasets we used so far would allow for a more comprehensive exploration of global password trends, reflecting diverse password requirements and user behaviors. In addition, while our current work focused on relatively small-scale cracking efforts and specific password structures, Phase 3 demonstrated the potential for large-scale password cracking. By refining our pattern-matching techniques and leveraging larger, more diverse datasets, we aim to scale our approach even further. However, the primary objective of this expansion is not just cracking passwords, but better understanding password vulnerabilities and offering stronger guidance for users to create more secure passwords.

Conclusion

Our project created a password analysis dashboard that delivers real-time feedback to help users strengthen their passwords. By leveraging advanced statistical, clustering and NLP methods, the analysis uncovered common patterns and vulnerabilities, underscoring the need for stronger password practices and offering valuable insights to enhance password security.